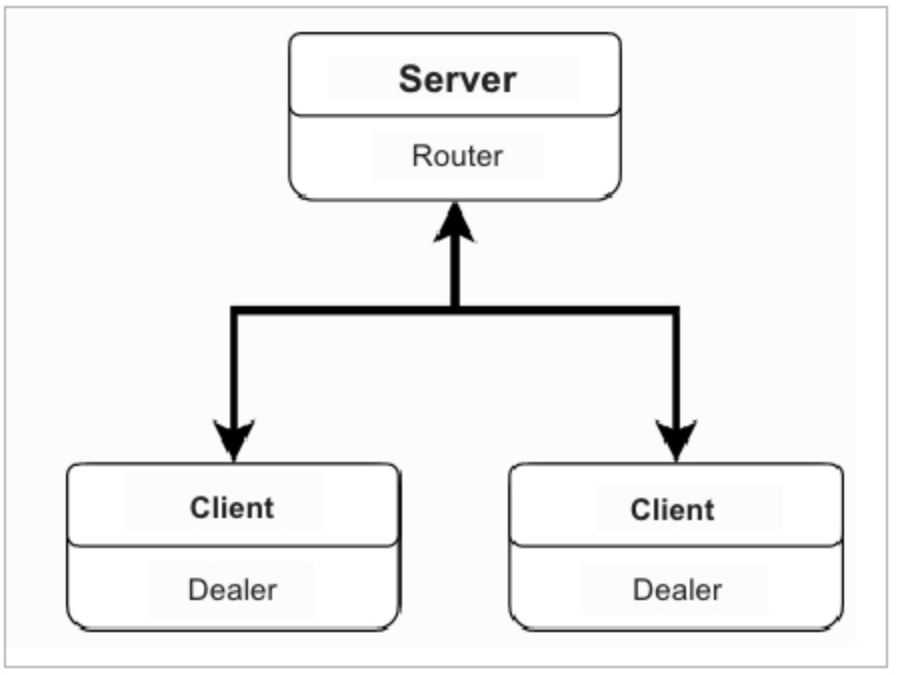

Figure 1: 0MQ router pattern

Connect is the mechanism for initiating the connection to the remote node. *Connect* performs a basic 0MQ *dealer-to-router* connection to the remote node and exchanges identity information for the purpose of supporting a two-way conversation. Connections sit atop 0MQ sockets and allow the *dealer/router* conversation.

Ping messages allow for keep-alive between *router* and *dealer* sockets.

Peer requests establish a bi-directional peering relationship between the two nodes. A *peer* request can be rejected by the remote node. If a *peer* request is rejected, the expectation is that a node attempts to connect with other nodes in the network via some strategy until the peering minimum connectivity threshold for that node is reached. If possible, the bi-directional relationship occurs over the already established 0MQ socket between *dealer* and *router*.

A *get_peers* message returns a list of peers of a given node. This can be performed in a basic connected state; it does not require peering to have occurred. The intent is to allow a node attempting to reach its minimum connectivity peering threshold to build a view of active candidate peers via a neighbour-of-neighbours approach.

An *unpeer* message breaks the peering relationship between nodes. This may occur in several instances, such as a node leaving the network. Nodes may also silently leave the network, in which case their departure will be detected by the failure of the ping/keep-alive message. An *unpeer* request does not necessarily imply a disconnect.

A *disconnect* message breaks the wire protocol connection to the remote node and informs the *router* end to clean up the connection.

Transmission methods

Transmission methods include the following.

Broadcast (msg) transmits an application message to the network following a 'gossipy' pattern. This does not guarantee 100 per cent delivery of the message to the whole network, but based on the gossip parameters, almost complete delivery is likely. A node only accepts messages for broadcast/forwarding from peers.

Send (node, msg) attempts to send a message to a particular node over the bi-directional 0MQ connection. Delivery is not guaranteed. If a node has reason to believe that delivery to the destination node is impossible, it can return an error response. A node only accepts a message for sending from peer nodes.

A *request (msg)* is a special type of broadcast message that can be examined and replied to, rather than forwarded. The intent is for the application layer to construct a message payload which can be examined by a special request handler and replied to, rather than forwarded on to connected peers. If the application layer reports that the request can't be satisfied, the message will be forwarded to peers as per the rules of a standard broadcast message. A node only accepts request messages from peer nodes.

Peer discovery

A bi-directional peering via a neighbour-of-neighbours approach gives reliable connectivity, with messages delivered to all nodes > 99 per cent of the time, based on the random construction of the network. Peer connections are established by collecting a suitable population of candidate peers through successive *connect/get_peers* calls (neighbours of neighbours). The connecting validator then selects a candidate peer randomly from the list, and attempts to connect and peer with it. If this succeeds and the connecting validator has reached minimum connectivity, the process halts. If minimum connectivity has not yet been reached, the validator continues attempting to connect to new candidate peers, refreshing its view of the neighbours of neighbours if it exhausts candidates.

The network component continues to perform a peer search if its number of peers is less than the minimum connectivity. This component rejects peering attempts if its number of peers is equal to or greater than the maximum

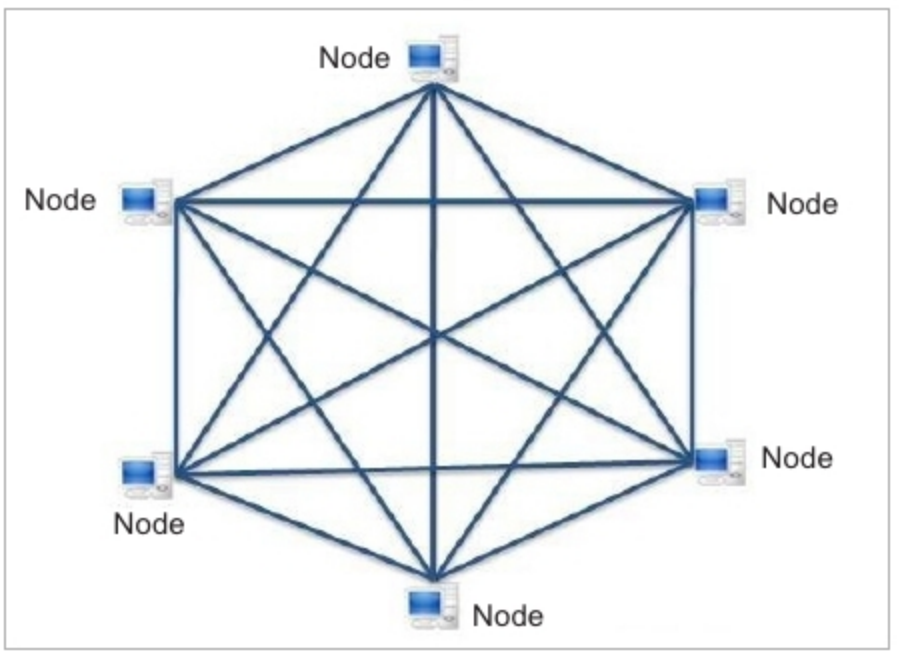

Figure 2: Bi-directional peering